

Interpretabilidad del Modelo

Entregable 3 - Proyecto Final

Objetivo. Explicar el comportamiento del modelo final entrenado con sklearn Pipeline, identificando las variables de mayor impacto, las variables descartadas, el efecto de la selección de características y el impacto de PCA sobre desempeño e interpretabilidad.

Elemento	Interpretacion
Modelo analizado	Pipeline con imputación, encoding, escalado, selección por varianza, PCA y Logistic Regression. El mejor modelo optimizado fue la Iteración 1, según la comparación por F1 Score y ROC-AUC.
Criterio de lectura	Se recomienda interpretar el modelo en dos niveles: importancia global con Permutation Importance sobre variables originales y coeficientes de Logistic Regression sobre componentes PCA.
Advertencia metodológica	Como PCA transforma las variables originales en componentes, los coeficientes finales no representan directamente variables originales. Por eso, Permutation Importance es la visualización principal para interpretar impacto por variable original.

1. Variables más importantes

La importancia debe evaluarse principalmente con Permutation Importance aplicada al pipeline completo, usando datos de prueba no vistos. Esta técnica mide cuánto cae la métrica del modelo cuando se altera una variable original. Por lo tanto, permite interpretar variables antes de PCA y antes del encoding, manteniendo una lectura cercana al negocio.

Con base en la construcción del problema y del target proxy sales_drop_risk, se espera que las variables más influyentes están asociadas con deterioro de calificación, presión negativa de reservas y comportamiento mensual del producto. En particular:

- monthly_avg_rating: nivel promedio de satisfacción mensual; valores bajos elevan el riesgo.
- rating_change: caídas en calificación capturan deterioro reciente del producto.
- negative_rate_change: incrementos en tasa negativa reflejan aumento de insatisfacción.
- rolling_3m_negative_rate: resume presión negativa persistente en los últimos meses.
- monthly_negative_rate y monthly_negative_reviews: cuantifican la intensidad de reseñas negativas.
- negative_helpfulness_pressure: combina negatividad con utilidad percibida de las reservas, capturando presión reputacional.

Nota crítica. Algunas de estas variables también participaron en la definición del target proxy. Por ello, su importancia alta es esperable y debe reportarse como una limitación: el modelo aprende patrones muy cercanos a la regla usada para construir la variable objetivo, no ventas reales observadas.

Visualización recomendada: Permutation Importance

```
from sklearn.inspection import permutation_importance
import matplotlib.pyplot as plt
import pandas as pd

best_model = random_search_iteracion_1.best_estimator_
```

```

perm = permutation_importance(
    best_model,
    X_test_formal,
    y_test_formal,
    scoring="f1",
    n_repeats=20,
    random_state=42,
    n_jobs=-1
)

perm_importance = pd.DataFrame({
    "variable": X_test_formal.columns,
    "importance_mean": perm.importances_mean,
    "importance_std": perm.importances_std
}).sort_values("importance_mean", ascending=False)

display(perm_importance.head(15))

perm_importance.head(15).sort_values("importance_mean").plot(
    x="variable",
    y="importance_mean",
    kind="barh",
    xerr="importance_std",
    figsize=(10, 7),
    legend=False,
    title="Permutation Importance - Top 15 variables"
)
plt.xlabel("Caida promedio en F1 Score")
plt.tight_layout()
plt.show()

```

2. Variables eliminadas

El pipeline aplica selección de características mediante VarianceThreshold. Este paso elimina variables con varianza muy baja después de la imputación, encoding y escalado. En la práctica, suelen eliminarse columnas constantes, indicadores de nulos sin variación y categorías one-hot muy raras o sin presencia suficiente.

Estas variables eliminadas no aportan señales útiles al modelo y pueden aumentar ruido, dimensionalidad y riesgo de sobreajuste. El siguiente código permite listar las variables retenidas y eliminadas después del encoding y antes de PCA:

```

pipe = random_search_iteracion_1.best_estimator_
encoder_step = pipe.named_steps["encoding"]
selector_step = pipe.named_steps["feature_selection"]

feature_names = encoder_step.get_feature_names_out()
selected_mask = selector_step.get_support()

variables_seleccionadas = pd.Series(feature_names[selected_mask], name="variables_seleccionadas")
variables_eliminadas = pd.Series(feature_names[~selected_mask], name="variables_eliminadas")

print("Variables antes de seleccion:", len(feature_names))
print("Variables seleccionadas:", len(variables_seleccionadas))
print("Variables eliminadas:", len(variables_eliminadas))

display(variables_eliminadas.head(30).to_frame())

```

3. Impacto de la selección de variables

Impacto	Explicación
Reducción de ruido	Al eliminar columnas casi constantes, el modelo evita aprender señales incidentales o irrelevantes.
Mejor estabilidad	Reduce la sensibilidad del modelo a categorías escasas o columnas con información insuficiente.
Eficiencia computacional	Disminuye la cantidad de variables que pasan a PCA y Logistic Regression.
Interpretabilidad parcial	Permite documentar qué variables fueron descartadas antes de la reducción dimensional.

Visualización recomendada: variables antes vs. después

```
conteo_variables = pd.DataFrame({
    "etapa": ["Antes de seleccion", "Despues de seleccion"],
    "cantidad_variables": [len(feature_names), int(selected_mask.sum())]
})

display(conteo_variables)
conteo_variables.plot(
    x="etapa",
    y="cantidad_variables",
    kind="bar",
    legend=False,
    title="Impacto de la seleccion de variables",
    figsize=(6, 4)
)
plt.ylabel("Cantidad de variables")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

4. Impacto de PCA

PCA se utiliza como reducción dimensional después de la selección de características. Su principal impacto es transformar un conjunto amplio y potencialmente correlacionado de variables en componentes principales no correlacionados. Esto ayuda a estabilizar Logistic Regression y puede mejorar la generalización, especialmente cuando hay muchas variables creadas por One-Hot Encoding.

La desventaja es que PCA reduce la interpretabilidad directa: cada componente es una combinación lineal de muchas variables. Por eso, los coeficientes del modelo final deben interpretarse como importancia de componentes, no como importancia directa de variables originales.

Visualización recomendada: varianza explicada por PCA

```
pca_step = random_search_iteracion_1.best_estimator_.named_steps["dimensionality_reduction"]

pca_summary = pd.DataFrame({
    "componente": [f"PC{i+1}" for i in range(len(pca_step.explained_variance_ratio_))],
    "varianza_explicada": pca_step.explained_variance_ratio_,
    "varianza_acumulada": pca_step.explained_variance_ratio_.cumsum()
})

display(pca_summary.head(15))

pca_summary.plot(
    x="componente",
    y="varianza_acumulada",
```

```

        kind="line",
        marker="o",
        figsize=(10, 5),
        title="PCA - Varianza explicada acumulada"
    )
    plt.axhline(0.85, linestyle="--", color="red", label="Umbral seleccionado")
    plt.ylabel("Varianza acumulada")
    plt.xticks(rotation=90)
    plt.legend()
    plt.tight_layout()
    plt.show()

```

5. Coeficientes del modelo

Como el modelo final es Logistic Regression después de PCA, sus coeficientes indican que los componentes principales aumentan o reducen la probabilidad estimada de riesgo. Coeficientes positivos elevan la probabilidad de `sales_drop_risk=1`; coeficientes negativos la reducen.

```

model_step = random_search_iteracion_1.best_estimator_.named_steps["training"]
pca_step = random_search_iteracion_1.best_estimator_.named_steps["dimensionality_reduction"]

coeficientes = pd.DataFrame({
    "componente": [f"PC{i+1}" for i in range(len(model_step.coef_[0]))],
    "coeficiente": model_step.coef_[0]
})
coeficientes["abs_coeficiente"] = coeficientes["coeficiente"].abs()
coeficientes = coeficientes.sort_values("abs_coeficiente", ascending=False)

display(coeficientes.head(15))

coeficientes.head(15).sort_values("coeficiente").plot(
    x="componente",
    y="coeficiente",
    kind="barh",
    figsize=(8, 6),
    legend=False,
    title="Coeficientes Logistic Regression sobre componentes PCA"
)
plt.xlabel("Coeficiente")
plt.tight_layout()
plt.show()

```

6. SHAP opcional

SHAP puede utilizarse como análisis complementario, pero en este pipeline su lectura directa queda afectada por PCA. Si se aplica SHAP sobre el espacio transformado, se explican los componentes principales. Si se desea explicar variables originales, es preferible priorizar Permutation Importance sobre el pipeline completo.

```

# Opcional: SHAP sobre el espacio transformado, no sobre variables originales.
# Requiere instalar shap en el entorno.
import shap

pipe = random_search_iteracion_1.best_estimator_
X_train_transformed = pipe[:-1].transform(X_train_formal)
X_test_transformed = pipe[:-1].transform(X_test_formal)

explainer = shap.LinearExplainer(pipe.named_steps["training"], X_train_transformed)

```

```
shap_values = explainer(X_test_transformed)

shap.summary_plot(shap_values, X_test_transformed, show=True)
```

Conclusión de interpretabilidad

El modelo optimizado identifica el riesgo principalmente a partir de señales de deterioro en calificaciones y reseñas negativas. La selección de variables reduce columnas poco informativas, mientras que PCA mejora la estabilidad y reduce dimensionalidad a costa de interpretabilidad directa. Por esta razón, la interpretación recomendada debe combinar Permutation Importance para variables originales con coeficientes del modelo sobre componentes PCA. La principal limitación es que el target fue construido como proxy a partir de variables de reserva, por lo que algunas importancias reflejan la propia regla de construcción del riesgo.